Generative Specification: A Discipline of Derivability for the Stateless Reader

Author: Juan Carlos Ghiringhelli (PragmaWorks) **Version:** 4.0 · **Date:** July 2026

> **If you build with AI agents, this is the discipline that lets you describe a system precisely enough that the agent builds it correctly — and then regenerate it, verify it against a live runtime, and trust the result.** The scarce skill stops being writing code and becomes specifying it. What follows is the method, the seven-property test of whether a specification meets it, and the layered evidence that it works — in controlled experiments, in production, at the formal tier, and — observationally — in the hands of practitioners who had never seen it before. The promise is concrete: **you will build correct systems you can describe.**

Abstract

The dominant failure mode of AI-assisted software development is **architectural drift**: an AI agent, capable of generating a system in a single session, resolves a thousand implicit decisions — units, field ordering, layer ownership, error semantics — silently, drawing on everything it has read about how such systems are *usually* built. Each decision is locally reasonable; in aggregate, across session, team, and service boundaries, they diverge. The cause is structural, not a model defect: the AI is a **stateless reader** that begins each session with none of the memory, shared context, or ability to ask that a human colleague uses to compensate for an underspecified system.

Generative Specification (GS) is a **discipline of removal** in Robert C. Martin's precise sense — like structured, object-oriented, and functional programming, it is defined by a freedom it takes away. But where those disciplines constrained control flow, data access, and mutability *for a human reader*, GS constrains a different axis: it removes the freedom to leave architectural intent implicit, for a reader that cannot recover it — requiring a specification from which a stateless reader can derive correct output unaided. (The word *paradigm* is used below only in Martin's narrow, technical sense — a discipline of removal; we make no Kuhnian claim about the field, and no claim that GS *succeeds* its forerunners in a lineage rather than constraining an orthogonal axis — see §2.) The discipline is operationalized as **seven properties** (Self-describing, Bounded, Verifiable, Defended, Auditable, Composable, Executable), each named for a failure mode observed in production and scored on a 14-point rubric.

The method rests on three ideas that are the author's own contribution (§2.3): **the bridge** — why externalizing intent into structure actually produces correct derivation, and why the load moves to the model's stronger side; **the sentinel navigational tree** — how a session's context is bounded so it stays clean enough to derive from; and **phase collapse** — specification, implementation, and verification converging into a single derivation step once the specification is complete and the executor is capable. The rest of the discipline is the author's honest synthesis of an industry already converging toward these practices (§2.3).

We also propose a theoretical home for the discipline — the **pragmatic tier** of the semiotic hierarchy (Morris, 1938), the relation of signs to an interpreter that carries no context, a tier prior programming disciplines left vacant. We offer that placement as a conceptual contribution, distinct from and not

required by the empirical results: the discipline stands on what it measurably produces, whatever one calls it.

The empirical program is layered so that independent experiments establish distinct, individually-checkable claims: that GS-derived software is **measurably better-structured** (AX), **independently reproducible** (RX), **deployable and operable in production** (EX), **cheaper to retrieve from** (KX), and **derivable at the formal, machine-checkable tier** (ALX); two June-2026 single-shot experiments (MX, RND-1) — complete and reproducible at $n = 1$ — extend the program to model-agnosticism and corner-cutting. Six production projects supply the discovery evidence, and observational field corroboration (§5.3) shows the method transferring to practitioners on their own code; the controlled experiments carry the confirmatory weight. The exact figures are set out in §5.

For a working developer, the shape of the payoff is simple: describe the system well once, and the building, the regenerating, and the verifying stop being where your time goes. The next section names the constraint that makes this possible.

---

1. Introduction: The Stateless Reader

On September 23, 1999, the Mars Climate Orbiter completed a nine-month, 416-million-mile crossing and arrived within 26 kilometers of its intended trajectory — then entered the Martian atmosphere at the wrong angle and was destroyed in 57 seconds. The cause was not a bug in any module. One team reported thruster impulse in pound-force seconds; the flight computer expected newton-seconds. Both components were internally correct, and *no test caught it — the code compiled cleanly*. The failure lived only at the boundary: the seam where two internally-coherent systems had to agree on a shared language they were never required to write down. The contract had been **assumed, not written**. It cost $327.6 million and two years to produce.

Today an AI assistant produces an equivalent interface in thirty seconds. The code compiles, types check, unit tests pass — and embedded in the output is a set of implicit assumptions the model resolved silently. The Orbiter problem did not go away; the velocity of producing it increased by orders of magnitude.

This is not an argument against AI-assisted development. It is an argument for what it requires. The Jacquard loom did not eliminate weaving craft — it relocated it upstream, into the card, where the pattern could be expressed once and executed at machine speed. Software construction is undergoing the same relocation. An AI agent with command-line access can read a codebase, write tests, run migrations, and commit, all in one session, starting from nothing — a capability now measured at scale on benchmarks such as SWE-bench (Jimenez et al., 2024). The executor is extraordinarily capable. The open question is what governs it across the session boundaries it cannot see, the team context it was never given, and the decisions made in conversations it was not part of.

The answer is a specification precise enough that a **stateless reader** — one with no memory of your intentions, no shared context, no ability to ask a clarifying question — can derive correct output from it alone. That discipline is what this work defines.

Naming the reader as stateless is a small move with a large consequence. The condition is not new — every AI session has always begun from nothing — but it went unnamed, treated as an inescapable fact of the tools rather than a solvable constraint. The three failures practitioners report — no reliable

process, no warranted trust, unreliable results — are its symptoms. Naming the constraint is the precondition for removing them.

This work requires an AI agent with direct CLI access — not a chat assistant, but an agent that can read and write files, run tests, commit, and execute commands (Claude Code, Cursor, and agentic IDE modes qualify). The methodology does not apply to chat-based interfaces.

Contributions. (1) The **derivability obligation** — naming implicit-context removal as the defining structural constraint of AI-assisted development. (2) **A proposed theoretical placement** — situating the discipline in the pragmatic tier of the syntactic/semantic/pragmatic tripartition, offered as a conceptual lens distinct from (and not load-bearing for) the empirical claims. (3) The **seven-property rubric** — a teachable, scored instrument validated across the production projects. (4) **Phase collapse** — specification, implementation, and verification collapsing into a single derivation step when the spec is complete and the executor is capable. (5) **The bridge and its asymmetry** — the positive premise for why externalized intent is derivable, and why the load moves to the model's stronger bank. (6) **The read-asymmetry** — that a system is comprehended from its structural surface, not from all of its code (§4.1). (7) **Cost inversion and the token economics of authored structure** — tokens-per-correct-output, not tokens-generated, as the binding metric. (8) A **layered validation program** (AX/BX plus EX/KX/ALX/RX) in which independent experiments close distinct sources of circularity.

Of these, three are the genuinely novel, load-bearing contributions — **the bridge and its asymmetry**, **the sentinel navigational tree**, and **phase collapse** (§2.3) — the ideas the method turns on. The rest are named framings and coinages (the derivability obligation, the pragmatic-tier placement, the read-asymmetry, cost inversion and its token economics) and the validation program: the author's naming and synthesis of a practice the industry was already converging toward, offered honestly as such. §2.3 draws that line explicitly.

---

2. A Discipline of Removal, and Its Theoretical Place

A paradigm, in Robert C. Martin's precise sense, is defined by what it *removes* from programmer freedom. Structured programming removed `goto`. Object-oriented programming removed unconstrained access to internal data. Functional programming removed unrestricted reassignment. **Generative Specification removes the freedom to leave architectural intent implicit.**

This removal differs in kind, and in *axis*, from the earlier disciplines. Structured, object-oriented, and functional programming each constrained a property of the code itself — control flow, data access, mutability; GS constrains a property of the specification's *relationship to its reader*. The axes are orthogonal: a functional or an object-oriented program can be fully GS-compliant or wholly non-compliant, because derivability for a stateless reader is independent of which control-flow or data disciplines the code obeys. GS is therefore a peer of those disciplines in the *removal* sense, not a successor in a progression. And where they constrained freedoms for human readers who could still compensate for gaps through memory, collaboration, and institutional context, GS's removal is *absolute for its reader*: a stateless executor that begins each session with none of those recovery mechanisms. What prior disciplines made inconvenient, GS makes structurally absent — because the reader that would have compensated does not exist.

2.1 Structural Disciplines: The Raw Material

Before the discipline can be explained, its raw material has to be named. A **structural discipline** is any long-established engineering practice that forces intent out of a developer's head and into the durable structure of the artifact — into names, boundaries, types, contracts, and recorded decisions. The canon is familiar to every senior engineer:

- **Intentional (clean-code) naming** — a name states what a thing is and does, so the reader need not trace its uses to find out.
- **SOLID** — single-responsibility, open-closed, Liskov substitution, interface segregation, dependency inversion: forces roles and seams to be explicit.
- **Domain-driven design's ubiquitous language** — the code speaks the business's vocabulary, so meaning is not translated in someone's head.
- **Design by contract** — preconditions, postconditions, and invariants stated at the boundary rather than assumed.
- **Type-driven design** — making illegal states unrepresentable, so the type is the specification.
- **Test-driven development (TDD)** — the test, the executable contract, is written before the code, giving constant validation and making implicit contracts explicit.
- **Hexagonal (ports-and-adapters) layering** — the core's dependencies point inward and are declared, not discovered.

Each of these is **encoded veteran wisdom**: a practice distilled from decades of expensive failure, the shared knowledge senior engineers reach for precisely because they have seen what its absence costs. GS does not invent these disciplines. Its move is to **force** them — to make their presence the measurable, gated definition of a well-specified system — because, as §4 argues, they are exactly the structures a stateless reader needs in order to derive correctly. Everything in this paper stems from that: *we force the structural disciplines*. The bridge (§4.1) is why forcing them works.

2.2 Phase Collapse: What the Discipline Buys

The reason to force these disciplines is not tidiness. It is **phase collapse**. In conventional development, specification, implementation, and verification are three phases separated in time, owned by different moments and often different people, with hand-offs and drift at every seam. When the specification is complete enough for a stateless reader to derive from, and the executor is capable enough to derive, the three phases **collapse into a single derivation step**: you write the specification, the agent generates the implementation, and the verification harness proves it — in one motion, repeatable on demand. And the collapse is not confined to spec → implementation → verification: it extends to the whole **operational lifecycle**. Deployment and environment setup are derived from the specification too — when a task calls for synthetic data, for spinning up and configuring Docker instances, or even for training a model on data extractable from the web, all of it is driven from the same specification rather than hand-managed as a separate phase. What collapses is the entire chain from intent to a running, provisioned system. This is the differentiator. It is *why* getting the specification right is worth the front-loading: the specification is no longer a document that precedes the work; it is the artifact from which the work is regenerated. Phase collapse is introduced here because everything downstream — cost inversion (§4.2), specification bandwidth, the whole economic argument — is a consequence of it.

2.3 What Is Ours, and What Is the Field's

Honesty about provenance strengthens the parts that are genuinely new. Much of this discipline is the author's synthesis of an industry already converging toward it, and the paper says so plainly:

- **Specification-as-driver is not novel.** Spec-driven development predates AI assistants by decades; it is the everyday practice of any team that writes the design before the code. GS's contribution is not the idea that the spec comes first.
- **The stateless-reader condition is not the author's discovery** — every practitioner has lived with it. What is the author's is the **naming and the wrapping**: turning an unexamined fact of the tooling into a named, solvable constraint with a discipline attached.
- **The structural disciplines** (§2.1) are the field's — SOLID, DDD, design by contract, and the rest are prior art. GS's move is to *force* them for a new reason.
- **The compact-knowledge-graph (CKG)** framing and term are **McCreary's** (Yarmoluk & McCreary, 2026), not the author's; GS borrows and generalizes it (§4.3).

Against that, the author's genuine contributions are three, and they carry the method:

1. **The bridge, and its asymmetry** (§4.1) — the positive premise for *why* externalizing intent into structure produces correct derivation, and why encoding intent in human-conceptual terms routes the hard half of the problem through the model's strong half.
2. **The sentinel navigational tree** (§4.1) — the concrete mechanism, a **canonical navigational tree (CNT) embedded in the agent's prompts**, that bounds a session's context so it does not degrade.
3. **Phase collapse** (§2.2) — the biggest differentiator, and the economic hinge of the whole method.

Alongside these sit a handful of smaller coinages inside larger topics — the **read-asymmetry** (§4.1), the naming of the stateless reader, the ratchet, the specification query — the author's contributions inside otherwise convergent territory. The map is deliberately modest: it is easier to trust a method whose author tells you which parts he invented.

2.4 The Theoretical Place

The semiotic tripartition — **syntactics, semantics, pragmatics** (Morris, 1938) — locates the discipline. Syntactic disciplines constrain the *form* of artifacts (what constructs are permitted, what dependency directions are allowed). Semantic disciplines constrain *meaning* (types, schemas, contracts). The **pragmatic** tier is the relation of signs to their interpreter in a context of use. GS occupies the pragmatic tier because it governs derivability for a reader who carries *no* interpretive context — the tier prior disciplines left vacant. Applying Morris's settled vocabulary to classify programming discipline is this work's proposal; the claim rests on the structural observation that no prior discipline stated the obligation to make the lifecycle layer derivable for a contextless executor. We present this placement as the work's conceptual contribution, not as a result to be defended empirically — the discipline and its measured effects in §5 stand whether or not a reader accepts the tier framing. The word *paradigm* is used here only in Martin's narrow, technical sense (a discipline of removal); we make no Kuhnian claim about the field, which the adoption data has not yet earned.

GS is also distinct from the spec-driven *tooling* now entering the mainstream — Amazon's Kiro and GitHub's Spec Kit, which scaffold a `requirements → design → tasks` document workflow for an agent. Those tools supply a *place* to write the specification; GS supplies the *obligation* the specification must

meet — derivability by a stateless reader — and a measurable test of whether it is met (the seven-property rubric). A Kiro `design.md` can be GS-compliant or wholly non-compliant; the workflow does not decide it. GS is the discipline a spec-driven workflow needs in order to be more than a folder of prose.

The Framework

Sections 3 and 4 together constitute the methodology: §3 defines the seven-property test of whether a specification meets the discipline, and §4 explains why meeting it produces correct derivation.

3. The Seven Properties

The discipline is operationalized as seven properties, each named for a class of failure observed in production, each scored 0–2 for a 14-point total. The rubric is the teachable spine of the method: a project is graded the way an AI reads it. Each property below is anchored to a **public, verifiable exemplar** — drawn from the open experiments or the public `pragmaworks` reference project, so a reader can inspect the evidence directly.

#	Property	What it removes	Public exemplar
1	**Self-describing**	Hidden purpose; the reader must infer what the system is	`pragmaworks` `CLAUDE.md` — a navigation root where every document location announces its domain (screaming architecture)
2	**Bounded**	Unbounded surface and context; the reader must scan everything	KX (`experiments/kx`): a routed navigation tree loads only the slice a query needs — measured cheaper *and* more accurate than dumping the whole codebase (macro-F1 0.808 vs 0.431)
3	**Verifiable**	Unchecked correctness; "it compiles" mistaken for "it works"	AX (`experiments/ax`): a Stryker mutation gate drove the mutation score from 58.6% to 93.1% MSI — proving *detection*, not merely line execution
4	**Defended**	Advisory rules the model treats as optional	AX (`experiments/ax`): Defended moved 0/2 → 2/2 only once gates were emitted as fenced file templates (the "First Response Requirements") — structural enforcement, measured
5	**Auditable**	Lost rationale; intentional decisions look like debt	`pragmaworks` ADR library (`docs/adrs/0001…0006` , each with context/decision/consequences) plus conventional-commit history
6	**Composable**	Tangled coupling that cannot recombine	AX (`experiments/ax`): interface-based dependency injection — the GS contribution over expert prompting — lets a stateless reader work a unit in isolation
7	**Executable**	Specifications that are never run against reality	EX (`experiments/ex`): Hurl probes + `slo-ramp-summary.json` — behavioral contracts run against a live runtime, not assumed from compilation

These properties are not independent virtues; they partition by **functional role** (§4.3). Two — Self-describing and Bounded — carry disproportionate weight, because a bounded, self-describing specification activates the model's relevant domain knowledge (a schema-fit effect: Bransford & Johnson, 1972) rather than its full prior distribution.

The Verifiable and Defended properties are not new instruments; they are the obligation to *apply* the standard quality toolchain and gate on it. The verification layer is assembled from type checkers, linters

(ESLint), test-coverage and cyclomatic-complexity gates, mutation testing (Stryker), dependency-vulnerability scans (`npm audit`), and quality-gate platforms such as SonarQube. GS does not reinvent these — it specifies which gates apply where and makes passing them the definition of *done* for a stateless executor that otherwise reports success on byte-write, not on correctness. Crucially, the project-specific gates are a **ledger of paid-for incidents**: each encodes a real production failure the discipline already paid for once — a field finding promoted to a named *forbidden pattern* and then to a versioned, shareable gate that carries the original incident as provenance. The rule set accumulates from observed failure, not opinion, which is what makes it cumulative and falsifiable. This is **the ratchet**: it does not reverse — every defect resolved becomes a test and a permanent rule, every ambiguity an ADR that closes a decision for good. Concretely, this is the routine that turns a single production incident into permanent grammar across three properties: the practitioner **adds a regression test** for the failure (closing **Verifiable**), **writes a post-mortem / incident record** capturing what happened and why (closing **Auditable**), and promotes the field finding to a **forbidden pattern → gate** that shrinks the reachable surface (closing **Bounded**). One incident, three durable artifacts, none of which reverses. A defect, in this frame, is not evidence the method failed but a **specification query** — *what constraint, had it been present, would have ruled this out?* — and once that constraint is written, the class of output that produced the defect becomes unreachable. Those same tools, being rubric-independent, double as external corroboration of the GS scores (§6).

The properties also generate a **diagnostic catalog**: their observable violations resolve into twenty-nine named pathologies (Architectural Drift, Session Amnesia, Implicit Contract Syndrome, and so on), each mapping to one or more absent properties and to the tier of the lifecycle cascade that structurally repairs it. The catalog is the rubric made actionable; it is developed in full in the Compendium.

3.1 The Seven Properties Cover Everything: A Concept Map

The method introduces several named concepts. They are not additions to the seven properties — each is *carried by* one or more of them. The map below is how the reader can see the seven cover the whole discipline.

Concept	Carried by (properties)	How
The bridge (§4.1)	Self-describing, Composable	Structure that a human-fluent reader can cross in both directions is exactly what makes a spec self-describing and its parts recombinable.
The sentinel navigational tree (§4.1)	Bounded, Self-describing	Each node declares its own scope (Self-describing) and routes so a session loads only its slice (Bounded).
The read-asymmetry (§4.1)	Bounded, Self-describing	Comprehension comes from a small high-signal structural surface, not all the code — which is what Bounded + Self-describing provide.
Phase collapse (§2.2)	Verifiable, Executable	The loop closes in one step only because verification is machine-checkable against a running system.
The ratchet / specification query (§3)	Defended, Verifiable, Auditable	Each resolved defect becomes a permanent gate (Defended, Verifiable) with recorded rationale (Auditable).
Cost inversion / near-reconstructability (§4.2)	Auditable	The document cascade and recorded decisions are what let a compliant codebase reconstruct its own spec.
Generative execution (§4.3)	Executable, Verifiable	The verify step — the full pyramid plus multimodal QA against the live app.
Contract sufficiency (§4.2)	Verifiable, Composable	Convergence against a contract, not line-by-line inspection, presupposes verifiable and composable units.

4. Why It Works

4.1 The Bridge, the Sentinel, and the Read-Asymmetry

The negative premise of GS is the stateless reader (the problem). The **positive** premise — why externalizing intent into structure actually *produces correct derivation* — is the bridge.

The bridge. Every structural discipline (§2.1) is a **bridge between human conceptual language and executable code** — the framing Gordon (2024) calls the *linguistics of programming*. Intentional naming, the specification, SOLID, domain-driven ubiquitous language, type-driven design, design by contract: each encodes human meaning in a form the machine can act on, and machine behavior in a form the human can verify. These disciplines were built to carry intent across that gap for the *next human reader*. Skilled human programmers were always fluent on both banks — human-conceptual language and code — and that dual fluency *is* the craft; it is not new. What the transformer adds is that it is the **first tireless machine executor** with that same dual fluency, now available at scale: it can read intent encoded as structure and emit code that encodes intent, tirelessly and on demand. And — crucially — it **shares the human asymmetry**. Like a human, it comprehends an *explanation* of behavior far more reliably than it reconstructs that behavior from raw code, and code carries no record of the *historical reasons and context* behind a decision. This is exactly why GS supplies the executor a **bridge in plain text, inside the codebase** — intent, the *why*, and structure expressed as ADRs, Mermaid diagrams, DB/API schemas, or whatever conveys it — which serves the machine executor precisely as it serves the next human reader. GS works because it makes building and maintaining that bridge the primary act of development. This is why we force the structural disciplines: they *are* the bridge.

The asymmetry is the sharper claim. The training corpus is overwhelmingly natural language; code is a small, exact, fragmented slice — every language its own precise rules, one wrong token breaks it. The model is therefore far stronger on human-conceptual meaning than on exact code. The leverage of the disciplines is not merely that they connect both banks: it is that **encoding intent in human-conceptual terms routes the hard half of the problem (exact code) through the model's strong half (human meaning it is fluent in)**. The discipline moves load from the model's weak bank to its strong bank.

The read-asymmetry is the same asymmetry, seen from the reading side, and it does more work than its brevity suggests. You do not comprehend a system by reading all of its code and inferring behavior and interactions line by line. You comprehend it by reading its **structural surface — contracts, unit tests, interfaces, design patterns, ADRs** — a small, high-signal slice that states what the code does and why, so the behavior need not be reconstructed from the implementation. This is the read side of the bridge: the reader derives *understanding* from the surface, not from the code, exactly as the reader derives *output* from the specification. It is why **Bounded** matters (a smaller, higher-signal surface is a cheaper read), why the **sentinel** works (it routes the reader to the right slice of that surface), and why **KX** (§5.2) measures what it measures (navigating an authored surface beats searching the code). The read-asymmetry is part of derivability: a stateless reader that can derive understanding from the structural surface can be given a far smaller, far cleaner context and still derive correctly.

The sentinel, and why it keeps the reader from degrading. Derivability is the *goal* — a stateless reader must be able to derive correct output from the specification alone. But there is a *mechanism* question underneath it: an AI session's working context degrades as it fills — the more a session loads, the more its attention smears across irrelevant material and the less reliably it derives anything. Derivability in principle is worthless if the reader's context is too polluted to exercise it. The mechanism that keeps the context clean is the **sentinel navigational tree**: a hierarchy of specification files (a **canonical navigational tree, CNT**, embedded in the agent's prompts) in which each node declares its own scope and routes to its children, so a session loads only the slice the task needs instead of the whole corpus. Paired with a deliberately **bounded tool surface — a small number of MCP tools rather than a sprawling set** — it holds the working context small and high-signal. A well-formed tree carries five categories — architectural identity, standards, constraints and prohibitions, tool sequencing, and routing; **tool sequencing is the most commonly absent and the most consequential gap**, because a tree that lists tools without stating when to prefer one over another forces unreliable inference. The link is the point: **derivability is the what, the sentinel plus bounded context is the how.** The spec makes correct output *derivable*; the sentinel keeps the reader's context clean enough to actually *derive* it.

The walls are correctness. Picture the verification pyramid — unit at the base, then integration, then end-to-end, contract, and non-functional at the peak. In GS its *walls are made of correctness*, and what raises them is the structural disciplines: the encoded, hard-won wisdom of expert veterans, shown to the agent as the shape it must build within. GS does not ask the agent to be a veteran. It shows the agent what veterans already know — the bridge is how that wisdom crosses over — and gates the output against it.

4.2 Cost Inversion

In traditional development, implementation accumulates a sunk cost; when a specification conflicts with a built system, the rational response is to amend the specification, because the code is load-bearing and the specification is not. GS inverts this. When regeneration is near-free, implementation carries no sunk

cost: fix the specification and regenerate. Code becomes an implementation residue. **The scarce resource is no longer the ability to write code; it is the ability to specify correctly.** And specifying correctly is not a tooling skill — it is the hard-won experience of techniques, strategies, patterns, and concepts held *above* tools and syntax: the judgment of a serious practitioner about what a correct system *is*, which no framework or syntax fluency substitutes for. That capacity is the contended human value, and it is exactly what does not regenerate for free.

This raises an apparent contradiction worth resolving directly, because it is a feature in disguise. From an ordinary, **non-GS** codebase, the specification is *not* reliably recoverable: the decisions, the alternatives considered, and the accumulated rationale were never written down and resist reconstruction. But a codebase that **fully satisfies the seven-property rubric is near-reconstructable** — you can recover the specification from it, precisely because it is **Auditable** and carries the **document cascade** (ADRs, conventional commits, the navigation tree, the recorded rationale). Compliance is what closes the gap. So the two statements are not in tension: *the spec is unrecoverable from a non-GS codebase; a fully compliant one is near-reconstructable* — and that near-reconstructability is exactly what the discipline is *for*. It is a feature. The more of the seven properties a codebase earns, the less its specification depends on a separate document to survive.

At portfolio scale this compounds into **specification bandwidth** — the binding constraint shifts from execution capacity to the rate at which intent can be correctly externalized into a durable specification. A project in a waiting state (deploy running, output under review) demands no execution from the practitioner, so portfolio size is bounded by status management, not execution load. And because the specification certifies *what* a valid implementation state is while the executor supplies the *how* within that boundary, the correctness criterion becomes **convergence, not inspection** — what we call **contract sufficiency**: the reader navigates and verifies against the contract instead of reading the derivation line by line.

A directional model formalizes this: $\mathbf{I \propto (1-S)/S}$, where S is specification completeness — the fraction of the output space the specification closes — and I is the expected number of correction iterations. It is a *mental model, not a formal result*: no units, no proportionality constant, no magnitude prediction. What it communicates is direction — each freedom the specification leaves unclosed is an additional correction cycle, and the cost rises sharply as S falls toward zero. The AX series gives cross-condition directional support (3/14 → 14/14 across eight conditions as S rises). Full treatment in Compendium §9.4.

4.3 Token Economics and the Discipline-Role Taxonomy

The one substantive objection GS meets in the field is token expenditure: writing the specification, the cascade documents, and the tests, then regenerating, all appear token-expensive. The honest reconciliation has two parts. First, the binding metric is **tokens-per-correct-output**, not tokens-generated: without authored structure, a session burns tokens on discarded wrong code, re-explained context every cold start, and drift repair. Second — and observed directly in the field — under inversion of control the *absolute* spend rises because the practitioner advances faster; **what is purchased is time, not tokens.** Both are true and not in tension. The per-correct-output economy is a *reasoned argument*, not an end-to-end measurement: KX (§5.2) measures the retrieval component directly — authored structure navigated rather than re-derived — but the full-session figure is not measured here. A widely-circulated ~70% token-reduction claim was conceded unproven in the field (§6; see also Experiment Supplement §S13) and is deliberately *not* asserted; the defensible claim is the KX-measured retrieval economy plus the reasoned downstream argument.

The mechanism is retrieval economics. An authored specification is a member of the **compact-knowledge-graph (CKG)** family — a small, enumerable, closed-vocabulary structure the reader navigates instead of re-deriving from prose at every query. The term and the framing are **Dan McCreary's** (Yarmoluk and McCreary, 2026), who benchmark it directly for knowledge retrieval: a pre-authored CKG costs ≈11× fewer tokens at ≈3.8× higher accuracy than chunked-prose RAG or query-time graph extraction, concluding that "when expert structure is available, the dynamic extraction step is wasted computation." GS borrows the framing and generalizes the CKG's single prerequisite relation to the executor's full operating path: navigation across cascade documents, dependency direction, applicable disciplines, and inviolable constraints.

This frames a GS session as **retrieve → generate → verify**. The read side is a retrieval problem — the read-asymmetry (§4.1) made operational — attacked from both ends: authoring the structure (the navigation tree) *and* shaping the code to be retrievable (the disciplines), with a code-search engine as the traversal. The harness is the categorically distinct *verify* step — **generative execution**: the full test pyramid (unit, integration, E2E, mutation, contract, NFR) plus multimodal-AI-as-QA exercised against the live application, not assumed from compilation — which checks output against the specification. GS is therefore retrieval-augmented *and verified* generation, with the retrieval **authored rather than inferred**.

This sorts the seven properties by **functional role**:

- **Verify** (secure correctness): TDD, BDD, design by contract, type-driven design, and the enforcement gates → Verifiable, Defended, Executable. Enforced by hooks, CI, and the harness.
- **Retrieve** (the bridge), in three sub-roles:
- *Legibility* — read the **what**: naming, SOLID interfaces, hexagonal layering, ubiquitous language → Self-describing, Composable. Enforced by the navigation tree and a structural code index.
- *Bounding* — leave **less** to read: DRY/deduplication, dead-code elimination, YAGNI, small units → Bounded. A second, independent lever on token cost (a smaller surface, not just better navigation), enforced by automated deduplication and dead-code detection over the dependency graph.
- *Decision-memory* — read the **why**: ADRs, conventional commits, engineering decision records → Auditable. Enforced by the cascade documents.

A discipline may serve more than one role; the grouping is by dominant function. Naming the role explains why the seven properties partition as they do — and why a codebase strong on verification yet weak on legibility and bounding still reads expensively.

A corollary sharpens the boundary with prompt engineering, and it is worth stating without hedging: **a complete specification makes prompt engineering unnecessary.** Prompt engineering is what you do when the instructions are incomplete — you coax, you demonstrate, you re-phrase until the model infers what you failed to state. A complete specification abstracts all of that *into the specification itself*: if it closes the output space, the stateless reader derives the correct output from the grammar alone, and there is nothing left for a clever prompt to add. A few-shot example is needed only where a constraint has not yet been stated — the example compensating for an incomplete specification, never improving a complete one. Prompt engineering is a symptom of an unfinished spec.

---

5. Evidence

The evidence falls into two categories of different epistemic weight. The **six production projects** are the substrate from which the methodology was *derived* — real systems built or refactored under increasingly rigorous discipline, surfacing a failure mode each and producing a corrective property. They carry the weight of discovery; built under early, still-maturing versions of the discipline, they are not the best demonstrations of it. The **six controlled experiments** are the *testing* phase, conducted after the methodology stabilized, with prospectively committed criteria designed to falsify rather than illustrate. The controlled results carry the confirmatory weight.

The controlled experiments at a glance. Each row's claim is independently checkable against the linked evidence; the final column states, in plain terms, what the result means for a practitioner.

Experiment	Claim it tests	Key result	What it means
AX	Output is measurably better-structured	3/14 → 14/14; 109 tests; construction-invariant (a tool-generated harness matched the best hand-built one)	The AI's output went from structurally broken to production-grade — and you don't need a guru to author the harness.
EX	Deployable and operable in production	13/13 behavioral probes (1,013 assertions), 6/6 SLO gates on a live runtime	The generated system actually shipped and held its performance targets under load.
KX	Authored structure is cheaper to retrieve from	macro-F1 0.808 vs 0.611 vs 0.431; up to 3.0× fewer tokens/query	Navigating an authored map is both more accurate and cheaper than dumping the code or searching it.
ALX	Derivability holds at the formal, machine-checkable tier	a compiler derived from its own specification; 386/386 acceptance tests	The method works even where "correct" means `cargo test`, not a rubric — a compiler built from its own spec.
RX	Independently reproducible	104 tests regenerated from the committed spec by a third party	Anyone with an API key can press a button and rebuild the result — it's not the author's word.
BX	The rubric is author-independent	$n = 3$; rubric rankings congruent with external metrics	The rubric measures something real, not just what its author wanted to see (preliminary).
MX *(single-shot, n = 1)*	The effect is model-agnostic	a mid-tier model matched a strong one, 149/149, at ≈6× lower cost	The gains come from the specification, not from paying for the biggest model.
RND-1 *(single-shot, n = 1)*	Prescriptive specification suppresses corner-cutting	descriptive 0/3 → prescriptive 3/3 at equal token cost	Tell the agent what you *mean*, not just what you want described, and it stops cutting corners — at the same cost.

5.1 Production Projects (Discovery)

The six production projects are the discovery substrate — real systems built or refactored under maturing versions of the discipline, each surfacing a failure mode that became a property: **SafetyCorePro** (Defended), **Invellum** (Bounded), **Conclave** (Composable), **BRAD** (Self-describing), **Shattered Stars** (Auditable), and a **Regulated Multi-Layer Data Platform** (Executable). They are **author-evaluated discovery substrate**, not controlled evidence; two (Conclave and the confidential client system informally referenced as *lumen*) are client-confidential and are not cited as public evidence, and ForgeCraft — the methodology's own tooling — is treated separately as the self-application

case, not a proof project. The confirmatory weight rests on the controlled experiments (§5.2), detailed next.

5.2 Controlled Experiments (Testing)

AX — the discipline produces measurably better-structured output (multi-agent adversarial study). Eight conditions (three pre-registered, five post-hoc) on the RealWorld Conduit benchmark, single practitioner, blind external rubric. The naive condition scored 3/14 (its test suites failed to compile); structured conditions reached the ceiling, with **two iterations (Treatment-v3, Treatment-v5) at 14/14**, 109 runner-verified tests (106 passing on independent re-run) against a live PostgreSQL database, and a documented **non-monotonic** path (a regression at v4 to 11/14, recovered at v5). A ninth condition tested **construction invariance**: a *tool-generated* harness (zero hand-tuning) reached the same ceiling as the best hand-built arm — 12-of-12 blind audit, 14/14 expanded scale, 11/11 live use-case probes. The structural advantage does not depend on expert hand-authorship. *What it means:* the same AI that produced structurally broken output under naive prompting produced production-grade structure under the discipline — and the method's benefit survives being automated, so it is not an artifact of one expert's touch. Evidence: `experiments/ax` (per-run JSON, rubric audits, generated harnesses).

EX — the result is deployable and operable in production. A Conduit backend specified, generated, verified, and deployed to a live cloud runtime (April 2026), the harness driving the full development→staging→production cycle. **Level 2:** 13/13 behavioral (Hurl) probes carrying 1,013 assertions, derived from use-case acceptance criteria, against the deployed service. **Level 3:** 3/3 environment-governance probes (configuration, secret hygiene, reachability). **Level 4:** 6/6 service-level-objective gates under load (aggregate p95 350 ms, p99 720 ms, error rate 0.04%). Fifteen defects were surfaced by failing probes and fixed before the cycle could close. This is the Executable property earned against a running system, not assumed from compilation. *What it means:* the specification did not just produce code that compiles — it produced a service that shipped, met its latency and error targets under load, and caught fifteen real defects on the way. Evidence: `experiments/ex` (Hurl probes, `slo-ramp-summary.json`, deploy logs).

KX — authored structure is cheaper to retrieve from (knowledge-retrieval replication). The CKG benchmark method run on a GS software harness across three conditions, each a fresh agent session per query: a routed navigation tree (CKG analog) reached macro-F1 **0.808 at 78.6k tokens/query**, beating both everything-in-context (RAG-dump: 0.611 at 100k) and code-search-at-query-time (no-structure: 0.431 at 233k) on accuracy *and* cost — up to 3.0× cheaper per query than the unstructured condition (233k → 78.6k tokens), and cheaper than the in-context dump as well. The architectural divergence replicated: aggregate-enumeration queries scored 0.909 (routed) vs 0.006 (no-structure), mirroring the original benchmark's 0.964 vs 0.054. The claim is deliberately **bounded**: *explicit structure beats inferred structure on structural queries*, not general superiority. *What it means:* giving the agent an authored map of the system, rather than the whole codebase or a search box, makes it both more accurate and up to three times cheaper to answer questions about that system — the absence of structure is the most expensive condition you can choose. Evidence: `experiments/kx` (per-query token counts, macro-F1 by condition); the full replication of the Yarmoluk–McCreary CKG benchmark on a GS software harness is developed in Compendium §7.8.E.

ALX — derivability holds at the formal, machine-checkable tier. *Loom is the formal-tier specification language of the GS ecosystem — a language precise enough that its own compiler can be derived from its specification; ALX is exactly that compiler, generated from Loom's spec alone*

(`github.com/jghiringhelli/loom`). A complete compiler for the Loom language was derived from its own formal specification alone (`cargo check` passed on first emission). The contribution is the correction curve from specification gaps to **S_realized = 1.0** (0.000 → 0.339 → 0.642 → 0.781 → 0.900 → 1.000 across six phases, terminating at **386/386** acceptance tests). Each correction was a *specification* improvement, not a code patch — enumerating exactly which classes of detail a formal specification must carry to be machine-derivable. This is spec derivability demonstrated at the machine-checkable layer above natural language, where conformance is `cargo test`, not a rubric. *What it means:* the discipline is not a trick of fuzzy natural-language grading — it holds where "correct" is decided by a compiler, all the way to a working compiler built from nothing but its own spec. Evidence: `experiments/alx` in the public Loom repository (spec, derived source, and correction log committed).

RX — the result is independently reproducible. From a single committed specification, any reader with Docker, Node, and an API key regenerates **104 passing tests across seven suites** (recorded run March 2026), with committed evidence (`jest-output.json`, `score.json`, build log) under `experiments/rx`. The result does not rest on the author's word; it is a button a third party can press. *What it means:* reproducibility here is literal — you can rebuild the whole result yourself from the spec, without trusting anyone.

BX — the rubric measures something independent of its author (blind cross-validation). Three community implementations never exposed to GS were scored on the rubric; the rankings were congruent with independent external metrics (CVE count, test count, type-health) — a preliminary cross-validation (n = 3) suggesting the rubric measures something that exists independent of its author, at feasibility-study scale rather than as a powered result. *What it means:* the rubric appears to track real structural quality, not the author's preferences — but with only three cases, this is a feasibility check, not a proof. Evidence: `experiments/bx` (per-implementation scores and external metrics).

MX and RND-1 — two June-2026 single-shot experiments (n = 1, complete and reproducible; weighted below the larger pre-registered set above for their scale, not for being preliminary, and reported as such). **RND-1** addresses the most common practitioner objection — that AI agents cut corners. Under explicit speed-and-token pressure, a *descriptive* specification let the model floor to the literal minimum (0/3 against a held-out acceptance oracle) while a *prescriptive* one — the same task, the intent made explicit — recovered the full intent (3/3) at equal token cost; separately, a stateless external judge confirmed that genuine reward-hacking (vacuous or improperly-mocked tests) did not survive independent verification — the reward-hacking failure mode now catalogued by ImpossibleBench, EvilGenie (arXiv:2511.21654), and the Reward Hacking Benchmark (arXiv:2605.02964). *What it means:* the complaint that "the agent fakes tests and does the minimum" is a specification-and-verification problem you can fix — make the intent prescriptive and let an independent judge verify — not an immovable property of the model, and fixing it costs no extra tokens. **MX** holds the GS specification constant and varies only the model: a mid-tier model matched a strong model at 149/149 on the full Conduit backend for ≈6× lower cost — evidence the discipline is **model-agnostic**, its effect a property of the specification rather than of any one model. *What it means:* the leverage is in the spec, so a cheaper model can do the same job — you are not buying results by buying the biggest model. Both are committed and reproducible — `experiments/rnd-1/` and `experiments/mx/`.

5.3 Field Corroboration (Observational)

A four-day GS workshop delivered to a paying eight-developer cohort at an insurance brokerage (June 2026) is reported as *observational* evidence, distinct from the controlled program (no control, no blind scoring, single self-selected cohort). It cannot test a hypothesis; it shows whether the controlled findings

recur when GS meets a real team on its own code — and it is the paper's evidence for **practitioner transfer**: eight developers who had never seen the discipline applied it, in a single engagement, to code they owned. That leg is observational, not a controlled result. On the greenfield day, every demonstrated project produced a running artifact within a single morning, and residual defects traced to *under-specification* or a missing asset rather than to the method — one participant diagnosing his own failed build without prompting: *"it is absolutely my fault, because I did not specify it correctly."* By the brownfield day every participant had mapped GS onto a production system, and one instituted a process change of his own accord, making an architecture decision record a merge prerequisite. The token objection recurred with a consistent trajectory — caution, an acute mid-workshop episode of exhausted budgets, then qualified willingness to adopt — corroborating the time-not-tokens reconciliation of §4.3. *What it means:* when the method met a real team on their own code, it behaved the way the controlled results predict — the failures were specification failures the developers could see and own.

---

6. Threats to Validity

The most serious risk is **guidance circularity**: GS guided the implementations and supplied the rubric. The validation program is layered to close it. BX applies the rubric to implementations it never guided and finds the rankings congruent with rubric-independent metrics — CVE count (`npm audit`), test count, type-health (`tsc`), and cyclomatic complexity. RX makes the executable result reproducible from archived artifacts by any third party. KX's structural-query ground truth derives from the same structure the navigation tree reads (the benchmark's own caveat), so its claim is bounded: *explicit structure beats inferred structure on structural queries*, not general superiority. Human-participant evidence is **observational only** (the McBrokers cohort, §5.3), not a controlled study. Results are **predominantly single-model (Claude)**; the AX series is **single-practitioner and directional** (eight conditions, non-monotonic, with a ninth testing construction invariance). The inverse-effort relation between specification completeness and correction cost is offered as a **directional model, not a formal result**. The ~70% token-reduction figure is **conceded unproven and deliberately not asserted**; the binding metric is tokens-per-correct-output, not tokens-generated. The six production projects are author-evaluated discovery substrate, not controlled evidence.

---

7. Implications for Practice

The specification precedes the code. The architectural constitution, structural diagrams, schema definitions, and at least a skeleton decision record must exist before the first agent session. This is not new; it was optional when the cost of skipping it was paid by a human who could compensate with memory. That compensation is unavailable to a stateless executor.

The document cascade. The individual artifacts below are established practice — every mature team writes ADRs, diagrams, and schemas. What GS changes is their *role*. They are not documentation *about* the code; they are the specification the code is *derived from* — authored first, the source of truth from which the implementation is regenerated (the inversion of §4.2). And together they are the **read-asymmetry (§4.1) made concrete**: the layered structural surface the stateless reader comprehends the system *through*, instead of reading the implementation. The novelty is the inversion and the read-surface, not the list of artifacts:

Document	What it specifies / is used for	Authored
Architectural constitution (the sentinel root — `CLAUDE.md` / `AGENTS.md` / `.cursor/rules`)	The grammar: identity, standards, inviolable constraints, tool sequencing, routing. Governs every session.	First — before any code
Sentinel navigational tree	The scoped child specs the root routes to, so a session loads only the slice its task needs.	With the constitution
Architecture Decision Records (ADRs)	The *why* — so the agent does not "correct" an intentional decision it lacks the context for.	At each decision; ongoing
Structural diagrams (C4)	The system at a glance — context for any agent entering the codebase.	Up front; revised on change
Use-case / sequence / state diagrams	Protocols (which calls, in which order), user journeys (which are also the E2E test scripts), and valid states/transitions (which also generate the state-test cases).	Before generating the behavior they describe
Schema definitions (DB / API / event)	The vocabulary of the system with its constraints formally stated.	Before implementation
Test suite (TDD / contracts)	The behavioral specification and a standing adversarial audit.	With each feature
Quality gates & commit hooks	Structural rejection of malformed output — the parser that makes certain mistakes unreachable.	Once; enforced continuously
Living (derived) documentation	Regenerated from the specs (OpenAPI, TypeDoc, Storybook) so it never drifts from the code.	Automatic — from the source

Specification-first, iterative delivery. GS is not a third methodology beside waterfall and agile. The specification layer runs waterfall — the grammar is written first; the delivery layer runs agile — each session produces atomic, tested, deployable commits. Waterfall's rigid front-loading dissolves because the constitution is a living document revised through commit discipline; agile's structural drift dissolves because the specification gates every session. Scope may be bounded: a complete specification of a minimum viable slice is still complete.

The industrial threshold. Three constraints historically prevented sustained formal discipline — learning (no career is long enough), maintenance (discipline erodes under deadline), and transfer (knowledge lived in people). A tireless executor that reads the specification before every session makes all three irrelevant. The practitioner's role shifts from implementation artisan to specification architect. The complexity moved upstream, into the card.

---

8. Related Work

Generative Specification sits in a lineage of work on instructing models and specifying software, but reduces to none of it. Each neighboring tradition supplies part of the picture; GS names the obligation they leave implicit — that a stateless reader must be able to *derive* correct output from the artifacts alone — and makes that obligation measurable.

Prompt engineering. The few-shot and chain-of-thought lineage (Brown et al., 2020; Wei et al., 2022) improves output by shaping the immediate input — demonstrations, worked reasoning, careful phrasing. GS treats these as compensations for an incomplete specification (§4.3): a few-shot example is needed exactly where a constraint has not been stated. Where prompt engineering optimizes the

transient prompt, GS invests in the durable, versioned specification from which any session derives — and argues that a complete specification makes prompt engineering unnecessary.

Context engineering and retrieval-augmented generation. RAG (Lewis et al., 2020) retrieves relevant text at query time to ground generation. GS is retrieval-augmented *and verified* generation with one decisive difference: the retrieval structure is **authored, not inferred** (§4.3). Rather than chunk-and-embed prose and hope the retriever surfaces the right span, GS has the engineer build a navigable structure — the sentinel tree, and the disciplines that make code retrievable — which the agent traverses. KX (§5.2) measures the payoff directly, and the CKG line of work it replicates (Yarmoluk & McCreary, 2026) shows authored structure beating query-time extraction.

Software specification theory. The obligation to state intent precisely is old. Parnas (1972) established information hiding and module specification — a module is defined by what a reader may assume, not by its implementation. Jackson's *Problem Frames* (2001) structure the relationship between a specification and the world it constrains. Meyer's design by contract (1992) makes preconditions, postconditions, and invariants first-class. GS inherits all three and adds the reader they did not name: each specifies *for a human engineer* who can still ask, remember, and infer. GS restates the obligation for a **stateless executor** that can do none of these, and turns "well-specified" from a judgment into a scored, gated rubric (§3).

AI code-generation tooling. Copilot-class assistants (Chen et al., 2021) and the agentic capability measured by SWE-bench (Jimenez et al., 2024) establish that the *executor* is extraordinarily capable — that a model can resolve real issues end to end. GS is orthogonal to executor capability: it governs what the capable executor is asked to build across the boundaries it cannot see. The spec-driven tools now entering the mainstream — Amazon's Kiro and GitHub's Spec Kit (§2.4) — supply a *place* to write the specification; GS supplies the *obligation* the specification must meet, and the test of whether it does.

Independent measurement of the problem. Two research efforts arrive at GS's problem statement from outside its frame, and neither reaches the paradigm claim — which is what makes them validation rather than corroboration. Orlanski et al.'s **SlopCodeBench (2026)** instruments agent trajectories directly: across 11 evaluated models extending their own prior solutions over 93 checkpoints, no agent solves any problem end-to-end, structural erosion rises in 80% of trajectories and verbosity in 89.8%, and agent code runs 2.2× more verbose than matched human-authored code while deteriorating with each iteration as human code stays flat; a prompt-intervention study improves initial quality but does not halt the degradation, leading the authors to conclude that "current agents lack the design discipline iterative software development demands." SlopCodeBench measures the failure mode independently — it does not test GS and is not cited as solution validation — but its result is decisive for §3's thesis: because prompting cannot arrest the erosion, the defect is **structural, not a model deficiency**, and only authored structure carried across sessions can prevent it. From a different angle, **Thirolf (2025, KIT/KASTEL)** independently identifies the same failure mode — implicit context that AI tools cannot access, observed through documentation-code traceability gaps in AI-assisted development — and proposes traceability tooling without arriving at GS: independent problem-identification, uncoordinated with this work. This convergence of independent lines of evidence is the strongest external validation available for the problem GS addresses.

Documentation-as-code. Treating documentation as a versioned, reviewed, CI-gated artifact alongside source (Gentle, 2017) is a direct ancestor of the GS cascade (ADRs, conventional commits, the navigation tree). GS sharpens the purpose: the cascade is not for the next human maintainer's convenience but is the **primary artifact from which the system is regenerated** — documentation promoted from description to source.

The through-line. Every neighbor treats derivability as an unstated background assumption — prompt engineering patches it per session, RAG infers it at query time, specification theory trusts a human reader to close the gaps, code-generation tooling measures the executor rather than the spec, and documentation-as-code describes a system built elsewhere. GS's contribution is to *name* the derivability obligation as the defining constraint of AI-assisted development, place it in the pragmatic tier (§2.4), and make meeting it measurable (§3) and falsifiable (§5). It is not a better prompt, a better retriever, or a better contract language; it is the discipline that names the property all of them were implicitly reaching for.

---

9. Conclusion

Architectural drift at generation speed is the anomaly that documentation-based convention cannot structurally prevent. The reconstitution is the specification becoming the primary artifact, with code as derived output, governed for a reader that carries no context of its own. The discipline is replicable (RX), measurable (the rubric, KX), and demonstrable in production (EX) and at the formal tier (ALX); observationally, it transfers to practitioners who meet it on their own code, and recurs in the field (§5.3). The specification is the mold. The AI is the foundry. The scarce resource — the one that does not regenerate for free — is the judgment to specify correctly.

For the developer who wants to build with agents, the invitation is direct. Learn to specify — to force the structural disciplines, to build the sentinel, to let specification, implementation, and verification collapse into one derivation — and the ceiling on what you can build alone rises to the rate at which you can describe it correctly. **You will build correct systems you can describe.** That is the promise, and the rest of this program is the evidence that it holds.

---

10. Lexicon of Coined Terms

The discipline introduces a vocabulary; these are the load-bearing coinages, collected for citation. Each is developed in the section noted (full glossary in the Compendium).

Term	Meaning	Developed in
Stateless reader	An executor that begins each session with no memory, no shared context, and no ability to ask — the reader GS specifies for.	§1
Architectural drift	The dominant failure mode: locally-reasonable decisions that diverge across session, team, and service boundaries.	§1, §3
Derivability	What a stateless reader can correctly determine from the artifacts alone; GS's binding constraint.	§1–2
Structural discipline	An established engineering practice (naming, SOLID, DDD, design by contract, type-driven design, hexagonal layering) that forces intent into durable structure — encoded veteran wisdom; GS forces these.	§2.1
The pragmatic tier	The semiotic level (signs to a contextless interpreter) GS occupies — the tier prior disciplines left vacant.	§2.4
Discipline of removal	A discipline defined by what it removes from programmer freedom (Martin's sense); GS removes implicit intent.	§2
Phase collapse	Specification, implementation, and verification converging in a single session when the spec is complete and gates close the loop.	§2.2
Schema-fit effect	A bounded, self-describing specification activates the model's relevant domain knowledge rather than its full prior.	§3
The ratchet	Accumulated tests, rules, and ADRs that do not reverse — each resolved defect a permanent constraint.	§3
Specification query	A defect reframed as the missing constraint that, had it been present, would have ruled it out.	§3
The bridge (asymmetric)	Structural disciplines bridge human language and code; the transformer is fluent on both banks, stronger on meaning — so intent is routed through its strong half.	§4.1
The read-asymmetry	A system is comprehended from its structural surface (contracts, tests, interfaces, patterns, ADRs), not from reading all its code — the read side of the bridge.	§4.1
Sentinel navigational tree	A scoped, routing hierarchy of specification files (a CNT in the agent's prompts) that bounds the session's context to the slice the task needs, keeping it from degrading.	§4.1
Cost inversion	When regeneration is near-free, the specification — not the code — becomes the scarce, load-bearing artifact.	§4.2
Specification bandwidth	The portfolio-scale binding constraint: the rate at which intent can be correctly externalized into a durable specification.	§4.2
Contract sufficiency	The spec certifies *what* a valid state is; the executor supplies the *how*; correctness is convergence, not inspection.	§4.2
Generative execution	The verify step: the full test pyramid plus multimodal-AI-as-QA exercised against the live application.	§4.3
Compact knowledge graph (CKG)	An authored, navigable structure the reader traverses instead of re-deriving from prose at every query. Term originates with Yarmoluk & McCreary (2026); GS borrows and generalizes it.	§4.3
Tokens-per-correct-output	The binding token metric, not tokens-generated.	§4.3

References (selected)

Full bibliography, glossary, the twenty-nine-pathology catalog, a practitioner protocol (under development), the formal-disciplines treatment, and the biological-isomorphism frontier are in the **Compendium**.

- Morris, C. W. (1938). *Foundations of the Theory of Signs*.
- Martin, R. C. *Clean Architecture* — paradigm as constraint.
- Bransford, J. D., & Johnson, M. K. (1972). Contextual prerequisites for understanding. *JVLVB*.
- Yarmoluk, D., & McCreary, D. (2026). *Benchmarking Knowledge Retrieval Architectures: RAG, GraphRAG, and Compact Knowledge Graphs*. v0.6.2 preprint. `github.com/Yarmoluk/ckg-benchmark`
- Gordon, C. S. (2024). *The Linguistics of Programming*. ACM SIGPLAN Onward! 2024 (Onward! Essays). https://doi.org/10.1145/3689492.3689806
- Jimenez, C. E., et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR.
- Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS.
- Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS.
- Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
- Parnas, D. L. (1972). *On the Criteria To Be Used in Decomposing Systems into Modules*. CACM.
- Jackson, M. (2001). *Problem Frames: Analyzing and Structuring Software Development Problems*. Addison-Wesley.
- Meyer, B. (1992). *Applying Design by Contract*. IEEE Computer.
- Chen, M., et al. (2021). *Evaluating Large Language Models Trained on Code* (Codex). arXiv:2107.03374.
- Gentle, A. (2017). *Docs Like Code*.
- Amazon (2025), *Kiro* (spec-driven agentic IDE); GitHub, *Spec Kit* — `requirements/design/tasks` agent workflows.
- Reward-hacking benchmarks: *ImpossibleBench*; *EvilGenie* (arXiv:2511.21654); *Reward Hacking Benchmark* (arXiv:2605.02964).
- Orlanski, G., et al. (2026). *SlopCodeBench: Benchmarking Agentic Code Quality Under Iterative Extension*. arXiv:2603.24755 [cs.SE].
- Thirolf, T. (2025). *Analysis of Project-Intrinsic Context for Automated Traceability Between Documentation and Code*. Bachelor's thesis, Karlsruhe Institute of Technology (KASTEL). https://mcse.kastel.kit.edu/downloads/theses/ba-thirolf.pdf
- NASA (1999). *Mars Climate Orbiter Mishap Investigation Board Final Report*.
- Experiment evidence (AX/EX/KX/ALX/RX): `experiments/` in the public repository, with committed per-run JSON.

Materials & Evidence

This is the white paper — the focused, publication-length statement of the methodology, intended for Zenodo deposit and journal submission. It is derived from the Compendium, the canonical ~115-page source that carries the full evidence, the failure-mode catalog, a practitioner protocol (under

development), the formal-disciplines treatment, and the biological-isomorphism frontier. Every claim and number here is traceable there; where this paper compresses, the Compendium expands, and both are kept consistent.

All materials are public. The Compendium and five of the six controlled experiments — **AX, BX, EX, KX, RX** — live in the public repository `github.com/jghiringhelli/generative-specification` (Compendium at `docs/white-paper/GenerativeSpecification_Compendium.md`; experiments, with per-run JSON evidence, under `experiments/`). The two June-2026 single-shot experiments — **MX** (`experiments/mx/`) and **RND-1** (`experiments/rnd-1/`) — are committed there with their specs, oracles, and generated outputs. The sixth experiment — the formal-tier **ALX** — and the **Loom** language itself live in the public Loom repository at `github.com/jghiringhelli/loom` (ALX at `experiments/alx`). The `pragmaworks` reference project is at `github.com/jghiringhelli/pragmaworks`.

Data Availability

All experiment evidence — per-run JSON, audit transcripts, coverage output, SLO summaries, and session logs — is public under `github.com/jghiringhelli/generative-specification/tree/main/experiments`, with the formal-tier ALX evidence under `experiments/alx` in the public Loom repository. Step-by-step replication instructions — clone, container setup, runner invocation, and blind-audit procedure — are in the **Experiment Supplement §S15**.